Practical No. 1

1. Aim

To define a simple service for converting Indian Rupees (Rs) into US Dollars (USD) and call it from different platforms like Java and .NET.

2. Prerequisites & Software Requirements

- **Code Editor:** Visual Studio Code.
- **API Testing Tool:** Postman.
- **Runtime Environments:** Node.js, Java Development Kit (JDK), and C# (.NET SDK).

3. Procedure

Project Setup Create a main folder named Cloud_Computing_Practical1 and set up the following internal folder structure:

- currency-conversion-service/ (Node.js Server)
- Java_Client/ (Java Application)
- NET_Client/ (.NET Console App)

Part A: Implementing the Node.js Web Service

Step 1: Initialize the Project Navigate to the `currency-conversion-service` folder, initialize npm, and install the required dependencies:

```
npm init -y
npm install express body-parser
```

Step 2: Create the Server Application Create a file named `server.js` and add the following code to define the RESTful API:

```
const express = require('express');
const bodyParser = require('body-parser');
const app = express();
const port = 3000;

app.use(bodyParser.json());

app.post('/convert', (req, res) => {
    const amountInRs = req.body.amount_in_rs;
    const conversionRate = 0.0145;
    const amountInUsd = amountInRs * conversionRate;
    res.json({ amount_in_usd: amountInUsd });
});

app.listen(port, () => {
    console.log(`Currency conversion service running on
http://localhost:${port}`);
});
```

Step 3: Run and Test the Server Run the server in your terminal:

```
node server.js
```

Test it using Postman by creating a POST request to http://localhost:3000/convert with the following JSON body:

```
JSON
{ "amount_in_rs": 1000 }
```

Part B: Implementing the Java Client

Step 1: Create the Java Application Navigate to the Java_Client folder and create a file named JavaClient.java with the following code to consume the Node.js service:

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class JavaClient {
    public static void main(String[] args) {
        HttpClient client = HttpClient.newHttpClient();
        String uri = "http://localhost:3000/convert";
        String requestBody = "{ \"amount_in_rs\": 1000 }";

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(uri))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(requestBody))
                .build();
```

```
        client.sendAsync(request, HttpResponse.BodyHandlers.ofString())
                .thenApply(HttpResponse::body)
                .thenAccept(System.out::println)
                .join();
    }
}
```

Step 2: Compile and Run Execute the following commands in the terminal:

```
javac JavaClient.java
java JavaClient
```

Part C: Implementing the .NET Client

Step 1: Initialize the .NET Project Navigate to the NET_Client folder and create a new console application:

```
dotnet new console
```

Step 2: Update the Application Code Replace the contents of Program.cs with the following code:

```
using System;
using System.Net.Http;
using System.Text;
using System.Threading.Tasks;

class Program {
    static async Task Main() {
        var client = new HttpClient();
        var apiUrl = "http://localhost:3000/convert";
        var jsonRequestBody = "{\"amount_in_rs\": 1000}";

        var response = await client.PostAsync(apiUrl,
            new StringContent(jsonRequestBody, Encoding.UTF8,
"application/json"));
```

```
if (response.IsSuccessStatusCode) {
            Console.WriteLine("Response Body: " + await
response.Content.ReadAsStringAsync());
        }
    }
}
```

Step 3: Run the Client Execute the program using the .NET CLI:

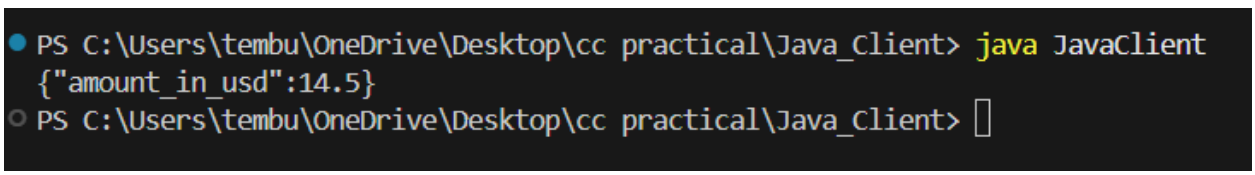
```
dotnet run
```

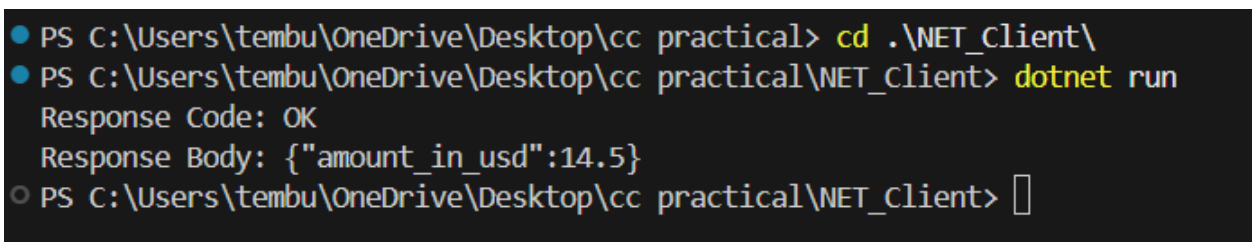
[Insert your screenshot here of the terminal showing the .NET output]

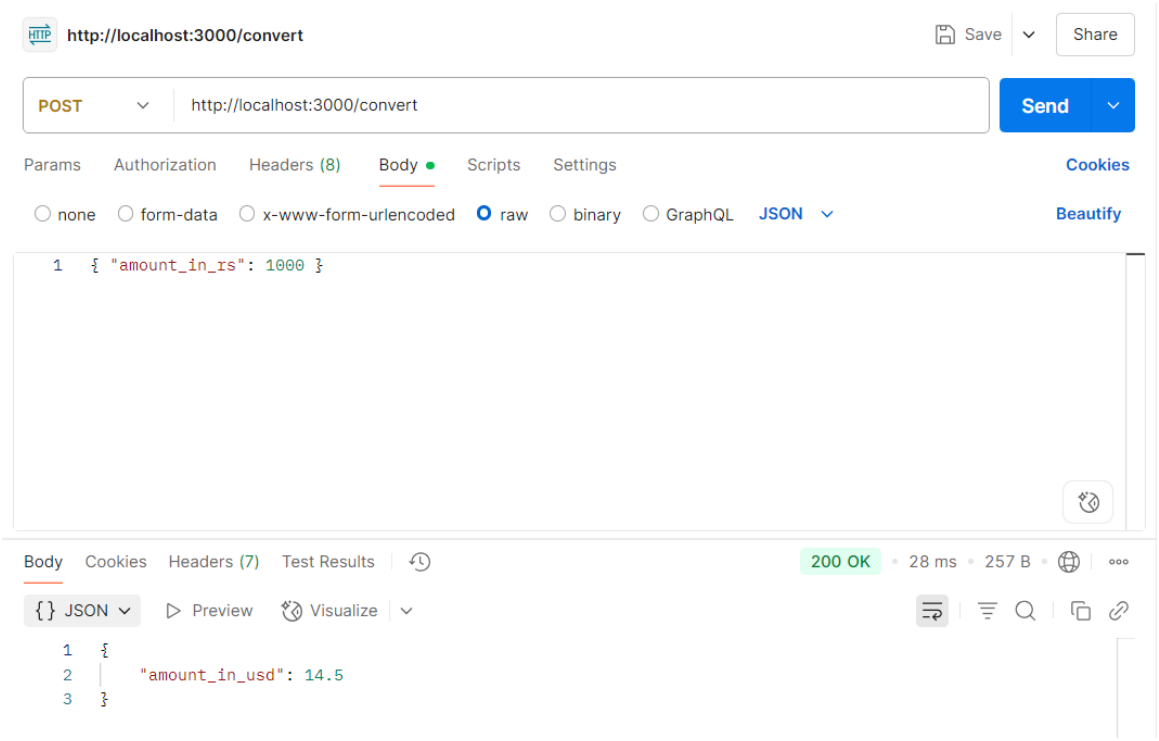
---

4. Conclusion

The practical successfully demonstrates a cross-platform architecture where a Node.js microservice provides a currency conversion API, which is then consumed by both Java and .NET clients using HTTP protocols.







Practical 2

1. Aim

To define and deploy a simple SOAP-based web service (Calculator Service) using Apache CXF and Maven, and to test it using a SOAP client.

2. Prerequisites

- **Java Development Kit (JDK)**: Installed and configured.
- **Apache Maven**: Installed and configured.
- **IDE**: VS Code, Eclipse, or IntelliJ IDEA.
- **Testing Tool**: SOAPUI or a web browser.

3. Procedure

Step 1: Create the Maven Project

Open the command prompt/terminal and generate a new Maven project using the quickstart archetype.

Command:

Bash

```bash
mvn archetype:generate -DgroupId=com.example -DartifactId=SimpleSOAPService
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

Step 2: Add Dependencies

Navigate to the `SimpleSOAPService` directory and update the `pom.xml` file to include the Apache CXF dependencies required for SOAP services.

File: `pom.xml`

XML

```xml
<dependency>
      <groupId>org.apache.cxf</groupId>
      <artifactId>cxf-rt-frontend-jaxws</artifactId>
      <version>3.4.5</version>
    </dependency>
    <dependency>
      <groupId>org.apache.cxf</groupId>
      <artifactId>cxf-rt-transports-http-jetty</artifactId>
      <version>3.4.5</version>
    </dependency>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-web</artifactId>
      <version>5.3.10</version>
    </dependency>
```

Step 3: Define the Service Interface

Create the interface that defines the methods exposed by the web service.

File: `src/main/java/com/example/calculator.java`

```java
package com.calculator;

import javax.jws.WebMethod;
import javax.jws.WebService;

@WebService
public interface CalculatorService {
    @WebMethod
    double add(double a, double b);
    @WebMethod
    double sub(double a, double b);
    @WebMethod
    double mul(double a, double b);
    @WebMethod
    double div(double a, double b);
}
```

Step 4: Implement the Service

Create the class that implements the logic for the interface methods.

File: src/main/java/com/example/calculatorImpl.java

```
package com.calculatorImpl;

import javax.jws.WebService;

@WebService(endpointInterface = "com.example.CalculatorService")
public class CalculatorImpl implements CalculatorService {

    @Override
    public int add(double a, double b) {
        return a + b;
    }

    @Override
    public int sub(double a, double b) {
        return a - b;
    }
    @Override
    public int mul(double a, double b) {
        return a * b;
    }
    @Override
    public int div(double a, double b {
        return a / b;
}
```

Step 5: Configure the Endpoint

Create a configuration file to wire the service bean and the endpoint.

File: `src/main/resources/cxf.xml`

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:jaxws="http://cxf.apache.org/jaxws"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://cxf.apache.org/jaxws http://cxf.apache.org/schemas/jaxws.xsd">

    <import resource="classpath:META-INF/cxf/cxf.xml" />

    <bean id="calculatorService" class="com.calculator.CalculatorImpl" />

    <jaxws:endpoint id="calculatorEndpoint"
                    implementor="#calculatorService"
                    address="/CalculatorService" />
</beans>
```

Step 6: Build and Run

Compile the project and start the service using the Maven exec plugin.

Commands:

```
mvn clean install

C:\ProgramData\chocolatey\lib\maven\apache-maven-3.9.12\bin\mvn.cmd
exec:java "-Dexec.mainClass=com.calculator.Server"
```

---

4. Output and Testing

A. WSDL Verification

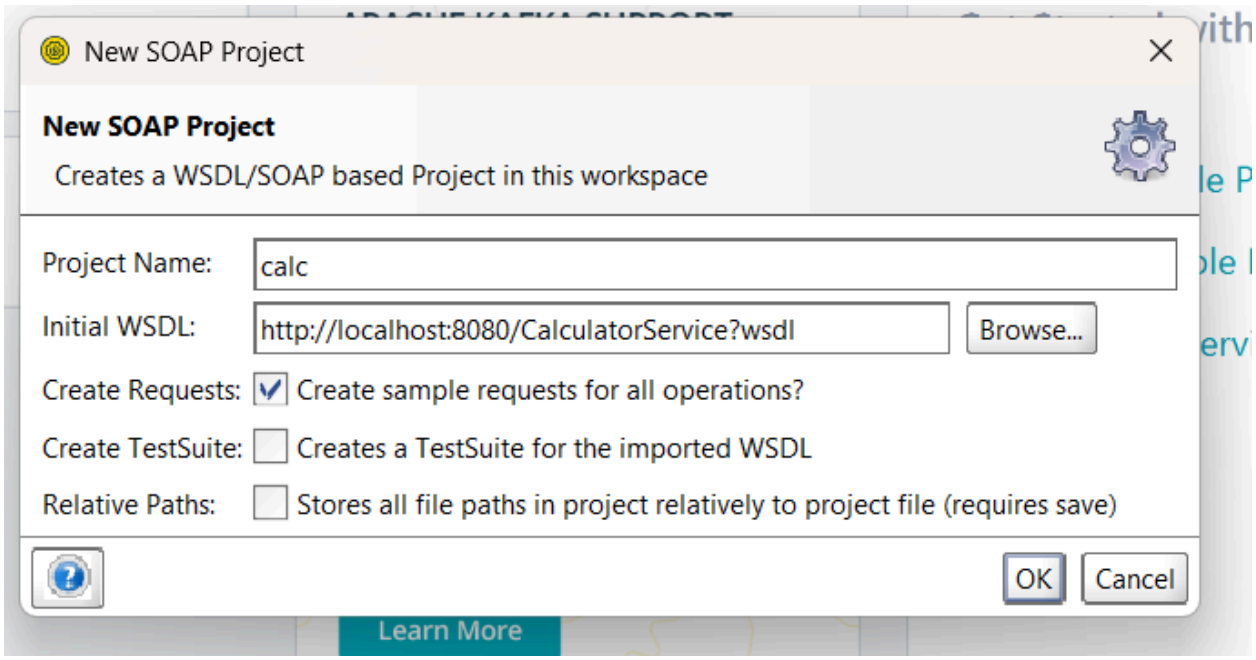
Open a web browser and navigate to the WSDL URL to verify the service is running.

- **URL:** http://localhost:8080/CalculatorService?wsdl
- **Observation:** The browser displays the XML WSDL definition containing `<wsdl:definitions>`, confirming the service is active.

B. Testing with SOAPUI

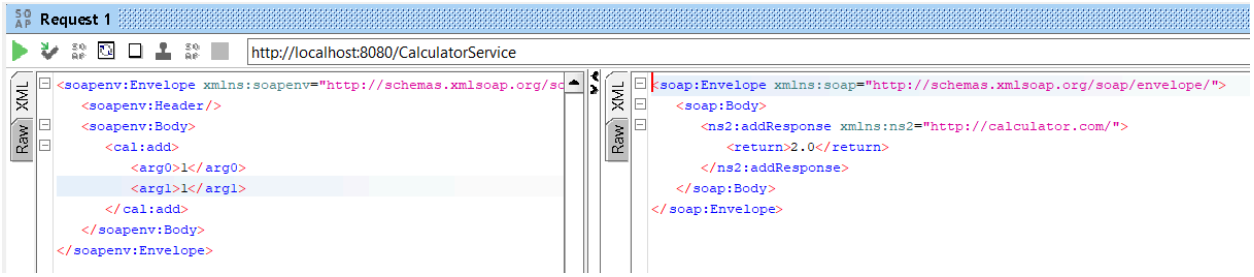
Step 1: Create a New Project

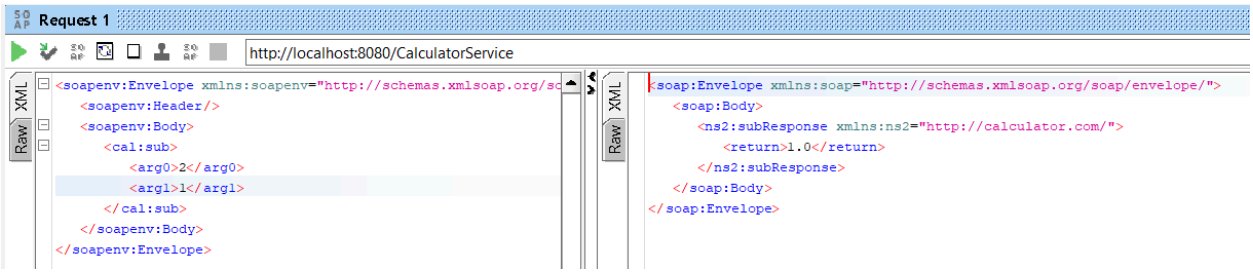
1. Click on the **"SOAP"** button (or go to **File > New SOAP Project**).
2. A small window will pop up asking for details.
3. **Project Name:** You can type `CalculatorTest`.
4. **Initial WSDL:** Paste the URL you found in the browser earlier: `http://localhost:8080/CalculatorService?wsdl`
5. Leave the other settings as they are and click **OK**.



Step 2: Send a Request

1. On the left side, you should now see a project tree (folders).
2. Click the small arrow to expand **CalculatorServiceSoapBinding**.
3. You will see your methods: **`add`** and **`subtract`**.
4. Expand **`add`** and double-click on **`Request 1`**.





Step 3: Test the Math

A window will open with XML code. Look for the ? characters inside the tags. Replace them with numbers.

Change this:

XML

```
<arg0>?</arg0>
<arg1>?</arg1>
```

To this (for example):

XML

```
<arg0>10</arg0>
<arg1>5</arg1>
```

Step 4: Run It

1. Click the **Green Play Button** (Triangle icon) in the top-left corner of that request window.
2. The right side of the window will show the response from your server.
3. You should see `<return>15</return>`.

5. Conclusion

We have successfully created and deployed a SOAP-based Calculator Web Service using Apache CXF and Maven. The service was verified by accessing its WSDL and performing functional testing using SOAPUI.

Practical 3

1. Aim

To design and implement a simple RESTful web service using Python and the Flask framework that creates, retrieves, and manages data using HTTP methods (GET and POST).

2. Prerequisites

- **Python**: Installed and configured.
- **Flask Framework**: Installed via pip (`pip install Flask`).
- **IDE**: VS Code or any text editor.
- **Testing Tool**: Web Browser and cURL (Command Line Tool).

3. Procedure

Step 1: Install Flask

Open the terminal and install the Flask library, which allows Python to run as a web server.

Command:

```
pip install Flask
```

Step 2: Create the Application File

Create a new file named `app.py` and import the required modules.

Step 3: Implement the REST API

Write the Python code to define the data source (a simple list) and the API endpoints (`/items`).

File: app.py

```
from flask import Flask, jsonify, request

app = Flask(__name__)
# Sample Data (Acting as a temporary database)
data = [
    {'id': 1, 'name': 'Item 1'},
    {'id': 2, 'name': 'Item 2'}
]
# Endpoint 1: GET All Items
@app.route('/items', methods=['GET'])
def get_items():
    return jsonify({'items': data})

# Endpoint 2: GET Single Item by ID
@app.route('/items/<int:item_id>', methods=['GET'])
def get_item(item_id):
    item = next((item for item in data if item['id'] == item_id), None)
    if item:
        return jsonify({'item': item})
    else:
        return jsonify({'message': 'Item not found'}), 404

# Endpoint 3: POST (Add) New Item
@app.route('/items', methods=['POST'])
def add_item():
    new_item = {
        'id': len(data) + 1,
        'name': request.json['name']
    }
    data.append(new_item)
    return jsonify({'message': 'Item added successfully', 'item': new_item}),
201

if __name__ == '__main__':
    app.run(debug=True)
```

Step 4: Run the Server

Execute the script in the terminal to start the REST service.

Command:

```
python app.py
```

Observation: The server starts running on http://127.0.0.1:5000.

---

4. Output and Testing

A. Testing GET Request (Retrieve Data)

Open a web browser or use curl to fetch the list of items.

- **URL:** http://127.0.0.1:5000/items
- **Response:**
- JSON

```
{
  "items": [
    {"id": 1, "name": "Item 1"},
    {"id": 2, "name": "Item 2"}
  ]
}
```

B. Testing POST Request (Add Data)

Use the command line to send a new item to the server.

- **Command:**

  For CMD:

```
curl -X POST -H "Content-Type: application/json" -d "{\"name\": \"New Item\"}" http://127.0.0.1:5000/items
```

  For PowerShell:

```
Invoke-RestMethod -Uri "http://127.0.0.1:5000/items” -Method POST -Body ‘{“name”:”new”}’ -ContentType “application/json”
```

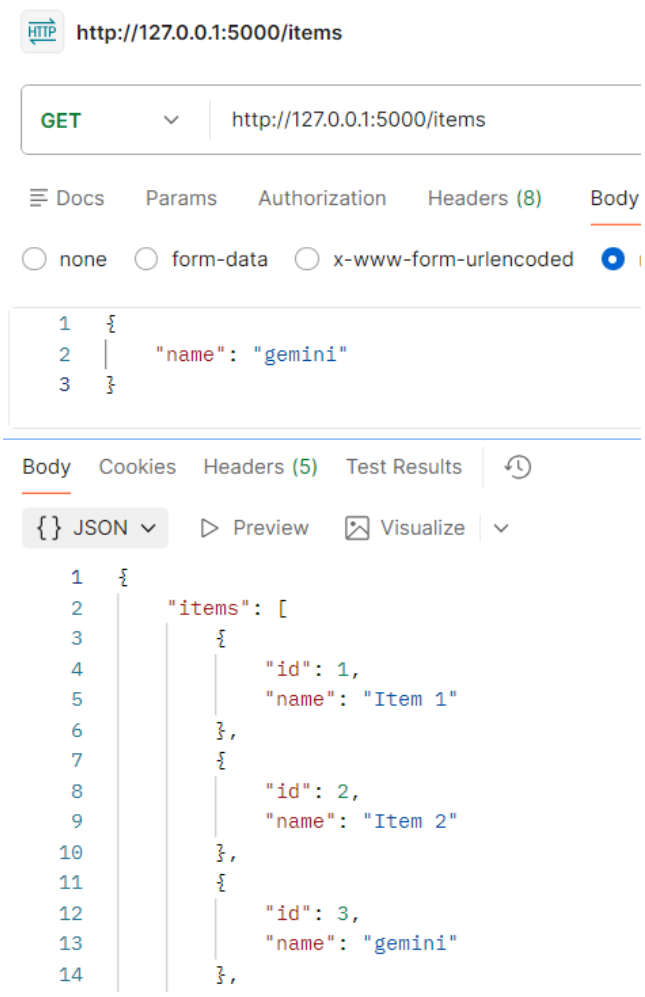
- **Response:**
- JSON

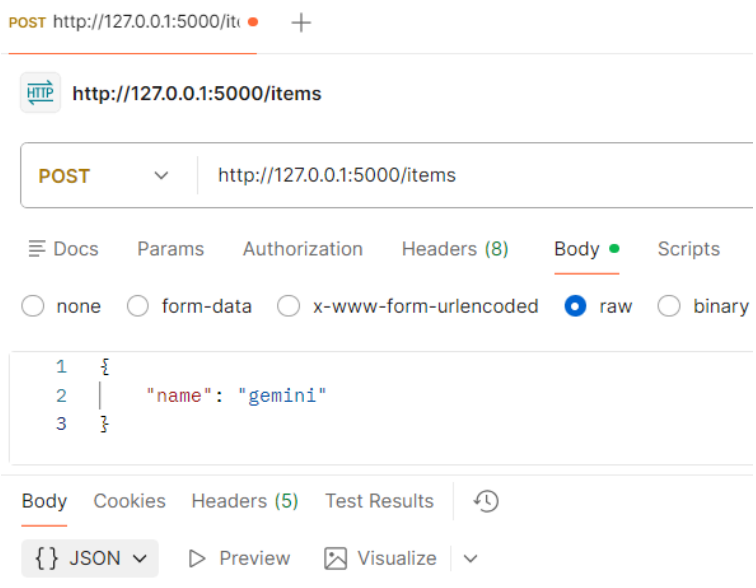
```
{
  "item": {
    "id": 3,
    "name": "New Item"
  },
  "message": "Item added successfully"
}
```

5. Conclusion

We have successfully implemented a RESTful web service using Python and Flask. We verified that the service can handle **GET** requests to display data and **POST** requests to accept and store new data in JSON format.

```
PS C:\Users\tembu> Invoke-RestMethod -Uri "http://127.0.0.1:5000/items" -Method Post -Body '{"name":"Gemini"}' -ContentType "application/json"

item                 message
----                 -------
@{id=5; name=Gemini} Item added successfully


PS C:\Users\tembu> Invoke-RestMethod -Uri "http://127.0.0.1:5000/items/5" -Method GET

item
----
@{id=5; name=Gemini}
```





Practical 4

1. Aim

To develop a Python application that consumes a RESTful Web Service (OpenStreetMap/Nominatim) to search for a location and retrieve its geographical details (Latitude, Longitude, and Address).

2. Prerequisites

- **Python**: Installed.
- **Requests Library**: A Python library for sending HTTP requests.

3. Procedure

Step 1: Install the Library

Open your terminal/command prompt and install the requests library. This allows Python to "talk" to websites.

Command:

```
pip install requests
```

Step 2: Create the Script

Create a new file named **map_search.py** and paste the following code.

Note on the Code: Unlike Google, OSM does not require an API Key. However, it **does** require a User-Agent header (a name for your app) so they know who is using their server.

File: map_search.py

```python
import requests
import json

def search_location(query):
    # 1. The OpenStreetMap (Nominatim) API Endpoint
    url = "https://nominatim.openstreetmap.org/search"

    # 2. Parameters for the search
    params = {
        'q': query,          # The place you are looking for
        'format': 'json',    # We want data in JSON format
        'limit': 5           # Limit results to top 5
    }

    # 3. Headers (Required by OSM policy)
    headers = {
        'User-Agent': 'StudentPracticalApp/1.0'
    }

    try:
        # 4. Send the GET request
        response = requests.get(url, params=params, headers=headers)

        # 5. Check if the request was successful
        if response.status_code == 200:
            return response.json()
        else:
            print(f"Error: API returned status code {response.status_code}")
            return None

    except Exception as e:
        print(f"Error occurred: {e}")
        return None

def main():
    print("--- OpenStreetMap Location Search ---")
    query = input("Enter a city or place name: ")

    print(f"\nSearching for '{query}'...")
    results = search_location(query)

    if results:
        print(f"\nFound {len(results)} result(s):\n")

        # Loop through the results and print details
        for index, place in enumerate(results, 1):
            print(f"Result #{index}")
            print(f"Name: {place.get('display_name')}")
            print(f"Type: {place.get('type')} ({place.get('class')})")
            print(f"Coordinates: {place.get('lat')}, {place.get('lon')}")
            print("-" * 40)
    else:
        print("No results found.")

if __name__ == "__main__":
    main()
```

Step 3: Run the Application

Save the file and run it in your terminal.

Command:

```
python map_search.py
```

4. Output and Testing

When you run the code, interact with it in the terminal.

Input:

```
Enter a city or place name: Mumbai
```

Expected Output:

```
--- OpenStreetMap Location Search ---
Enter a city or place name: Mumbai

Searching for 'Mumbai'...

Found 1 result(s):

Result #1
Name: Mumbai, Mumbai Suburban District, Maharashtra, 400051, India
Type: city (place)
Coordinates: 19.0549990, 72.8692035
----------------------------------------
```

5. Conclusion

We successfully developed a Python application that consumes the OpenStreetMap Nominatim REST API. We learned how to send HTTP GET requests with headers and parse JSON responses to extract geographical data without requiring a paid API key.

Practical 5

1. Aim

To install and configure hypervisor-based virtualization using KVM (Kernel-based Virtual Machine) on a Linux host system and deploy a guest operating system.

2. Prerequisites

- **Host Operating System:** Linux (Linux Mint / Ubuntu).
- **Hardware:** Processor with Hardware Virtualization support enabled (Intel VT-x or AMD-V).
- **Software:** `qemu-kvm`, `virt-manager` (Virtual Machine Manager).
- **Guest OS Image:** A bootable ISO file (e.g., TinyCore Linux).

3. Procedure

Step 1: Verify Hardware Virtualization Support

Open the terminal and check if the host CPU supports virtualization.

```
grep -E -c "vmx|svm" /proc/cpuinfo
```

(Observation: An output greater than 0 confirms that hardware virtualization is supported).

Step 2: Update System Repositories

Ensure the local package index is up to date before installing the required tools.

```
sudo apt update
```

Step 3: Install KVM and Management Tools

Install the core KVM engine, the libvirt daemon for managing virtual machines, and the graphical Virtual Machine Manager (`virt-manager`).

```
sudo apt install qemu-kvm libvirt-daemon-system libvirt-clients
bridge-utils virt-manager -y
```

Step 4: Configure User Permissions

Add the current user to the `libvirt` and `kvm` groups so that virtual machines can be managed without requiring root access every time.

```
sudo adduser $USER libvirt
sudo adduser $USER kvm
```

(Note: A system logout or reboot is required for these permission changes to take effect).

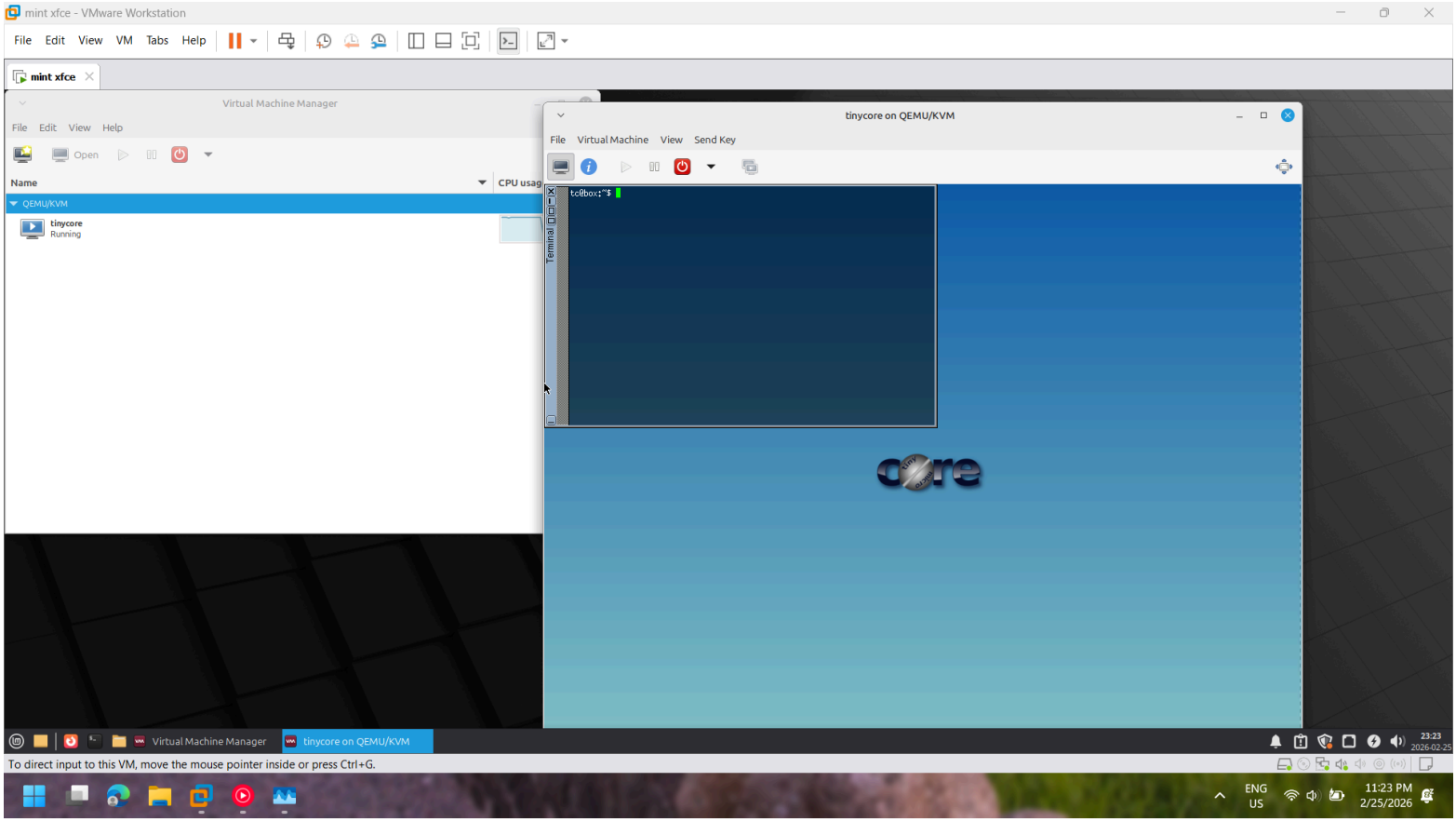
Step 5: Verify the Virtualization Service

Check the status of the libvirtd daemon to ensure the backend service is actively running.

```
sudo systemctl status libvirtd
```

Step 6: Create and Configure the Virtual Machine

1. Launch the **Virtual Machine Manager** (virt-manager) application from the application menu.
2. Click on **File > New Virtual Machine**.
3. Select **Local install media (ISO image or CDROM)** and click Forward.
4. Browse and select the downloaded Guest OS ISO (e.g., TinyCore Linux). Select "Generic default" for the operating system type if it is not automatically detected.
5. Allocate system resources. For a lightweight distribution, allocate **512 MB of RAM** and **1 CPU core**.
6. Uncheck "Enable storage for this virtual machine" to run it directly as a Live OS in memory, or allocate disk space if a permanent installation is required.
7. Name the virtual machine (e.g., TinyCore_KVM) and click **Finish**.



Practical No. 6

1. Aim

To develop a server-side application capable of receiving and saving an uploaded image (binary file) and to demonstrate the upload process using multiple client tools (Postman and PowerShell).

2. Prerequisites & Software Requirements

- **Runtime Environment:** Node.js
- **Backend Framework & Libraries:** Express.js, Multer (for handling multipart/form-data)
- **Client Testing Tools:** Postman, Windows PowerShell (v7+ recommended)
- **Code Editor:** Visual Studio Code

3. Procedure

Step 1: Initialize the Project and Install Dependencies

Create a new project folder named Practical_6_Upload and an empty subfolder inside it named uploads. Open the terminal in the main folder and initialize the Node.js project:

```
npm init -y
npm install express multer
```

Step 2: Implement the Upload Server

Create a file named server.js and write the following code to configure the server and define the file storage parameters using Multer:

JavaScript

```javascript
const express = require('express');
const multer = require('multer');
const path = require('path');

const app = express();
const port = 3000;

// Configure Multer storage engine
const storage = multer.diskStorage({
    destination: function (req, file, cb) {
        cb(null, 'uploads/'); // Directory to save uploaded files
    },
    filename: function (req, file, cb) {
```

```
        // Append current timestamp to maintain original file extension
        cb(null, Date.now() + path.extname(file.originalname));
    }
});

const upload = multer({ storage: storage });

// Define the POST endpoint for file uploads
app.post('/upload', upload.single('myImage'), (req, res) => {
    if (!req.file) {
        return res.status(400).send({ message: 'No file uploaded.' });
    }
    res.status(200).send({
        message: 'File uploaded successfully!',
        filename: req.file.filename
    });
});

app.listen(port, () => {
    console.log(`Image upload server running on http://localhost:${port}`);
});
```

Step 3: Start the Server

Run the server application from the terminal:

```
node server.js
```

Step 4: Test Upload via Postman (Client 1)

1. Open Postman and create a new **POST** request to http://localhost:3000/upload.
2. Navigate to the **Body** tab and select **form-data**.
3. Set the Key type to **File**, enter myImage as the Key, and browse for an image file on your local system as the Value.
4. Click **Send**.

Step 5: Test Upload via PowerShell (Client 2)

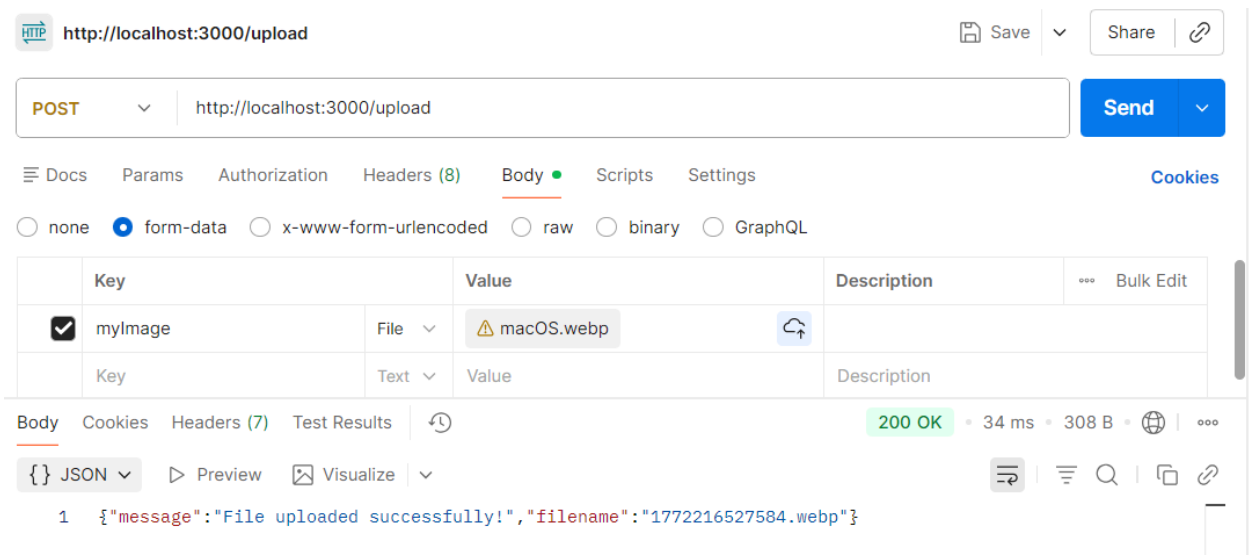
Alternatively, use PowerShell to script the file upload by utilizing the Invoke-RestMethod command to send a multipart form request to the server:

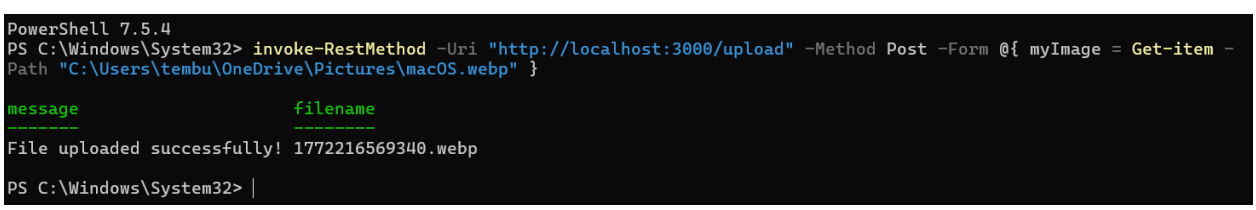
```
# Define the path to the local image
$imagePath = "C:\Users\YourUsername\Pictures\sample_image.png"

# Execute the REST POST request with the file payload
Invoke-RestMethod -Uri "http://localhost:3000/upload" -Method Post -Form @{
    myImage = Get-Item -Path $imagePath
}
```

---

4. Output / Observation

- **Server-Side:** The server successfully received the HTTP POST request. The image file was parsed and saved correctly inside the local `uploads/` directory with a timestamp appended to the filename.
- **Client-Side:** Both Postman and PowerShell received an HTTP 200 OK status code along with the JSON response confirming the successful file upload.





5. Conclusion

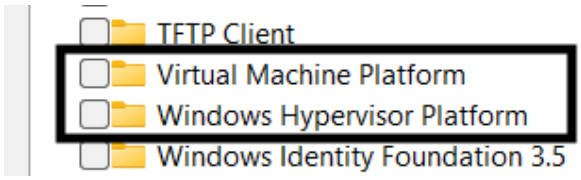
We successfully developed a Node.js web service to handle binary file uploads. We learned how to process `multipart/form-data` payloads using the Multer middleware and demonstrated that RESTful APIs can be consumed by various clients, including API testing suites (Postman) and command-line interfaces (PowerShell).

Practical No. 7

Aim: To implement Proxmox functionality to provide Virtual Server Infrastructure (VSI) and Infrastructure as a Service (IaaS) by configuring a cloud node and provisioning virtual machines.

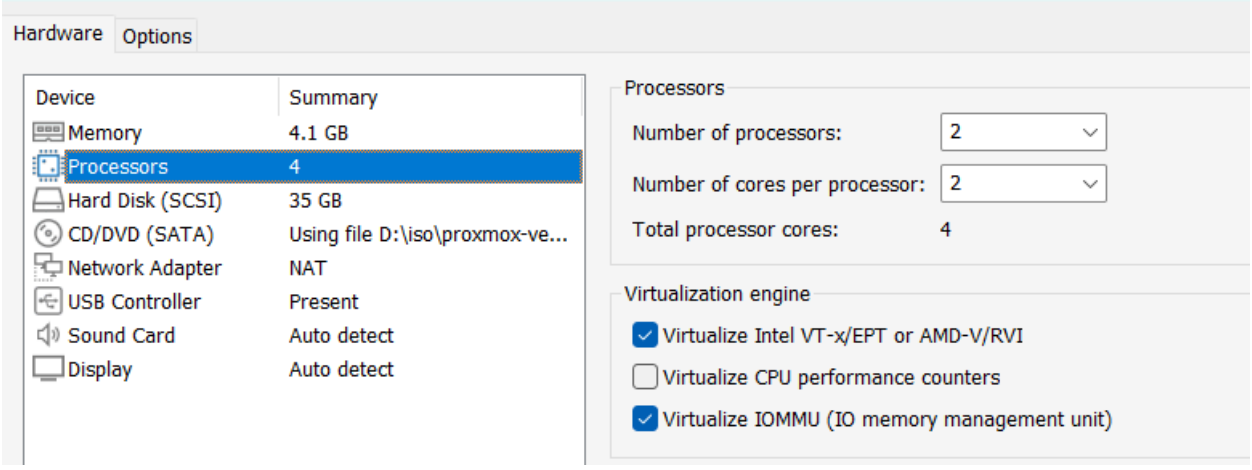
Prerequisites:

- VMware Workstation Pro
- Proxmox Virtual Environment (VE) ISO
- TinyCore Linux ISO
- In start menu search and go to “turn windows features on or off” and turn off these two features



Phase 1: Creating the Proxmox Virtual Machine in VMware

1. Open VMware Workstation Pro and choose to create a new virtual machine.
2. Select to install from the Proxmox ISO file.
3. Set the guest operating system version to **Debian 12.x 64-bit**.
4. Provide a name for the virtual machine.
5. Set the virtual disk size to **35 GB**.
6. Click on **Customize Hardware** to adjust the VM settings:
   - **Memory:** Set the RAM to **4.2 GB**.
   - **Processors:** Set to **2 processors** with **2 cores per processor**.
   - **Virtualization Engine:** Ensure both **"Virtualize Intel VT-x/EPT or AMD-V/RVI"** and **"Virtualize IOMMU"** are enabled.



7. Click **Finish** and then **Power on** the virtual machine.

Phase 2: Installing Proxmox VE

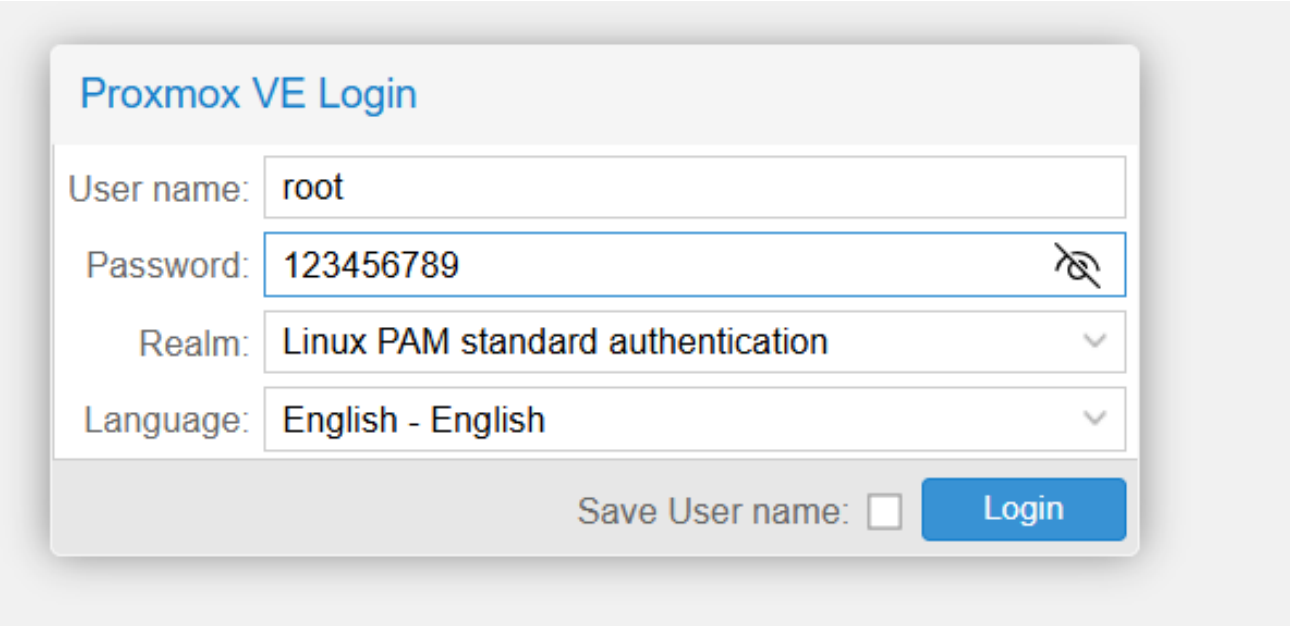
1. When the Proxmox boot menu appears, select **Install Proxmox VE (Graphical)**.
2. Proceed through the setup wizard.

3. Leave the disk settings at their defaults.
4. When prompted for a password, set it to **"123456789"**.
5. At the "Management Network Configuration" screen, keep the default settings (the system will populate these automatically).
6. Click **Next** and then **Install**. Wait for the installation to complete.
7. Once the installation is finished and the system reboots into the Command Line Interface (CLI), it will prompt you to log in.
8. The username is **"root"**, and the password is the one you set previously ("123456789"). *(Note: Linux CLI does not display password characters as you type)*.

```
-----------------------------------------------------------------------------

Welcome to the Proxmox Virtual Environment. Please use your web browser to
configure this server - connect to:

  https://192.168.160.129:8006/

-----------------------------------------------------------------------------

prox login: root
Password:
Linux prox 6.17.2-1-pve #1 SMP PREEMPT_DYNAMIC PMX 6.17.2-1 (2025-10-21T11:55Z) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@prox:~#
```

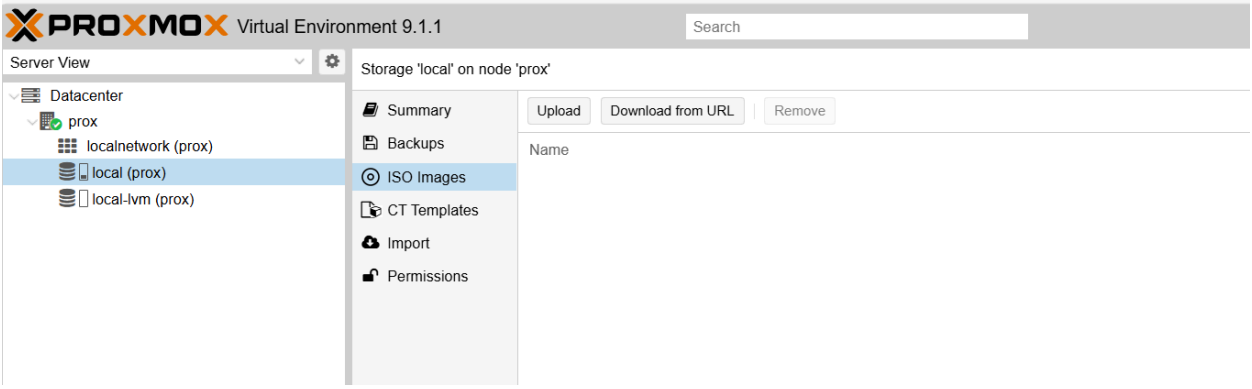
Phase 3: Accessing the Web Dashboard and Uploading the ISO

1. Open a web browser on your host machine and navigate to the IP address and port (usually https://[IP Address]:8006) displayed in the Proxmox CLI.
2. Log in to the Proxmox dashboard using the same credentials (**root** / **123456789**).

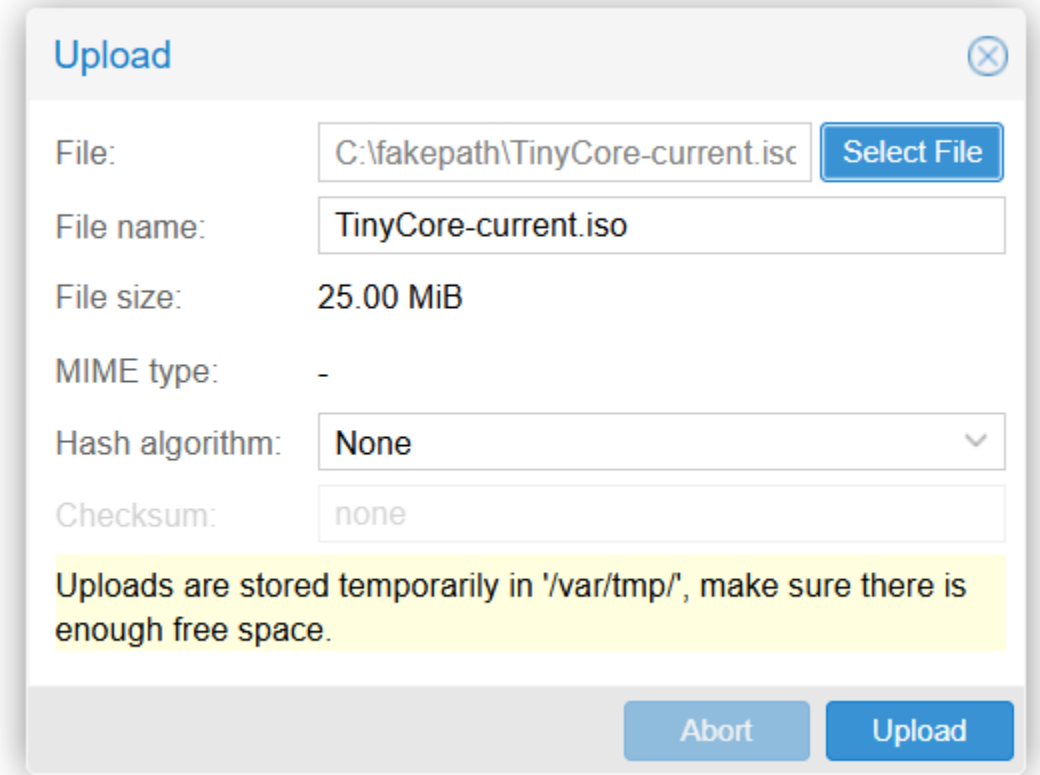


3. In the left-hand navigation pane, expand the node (e.g., prox) and click on **local (prox)**.
4. On the right-hand panel, select **ISO Images**.

5. Click the **Upload** button.



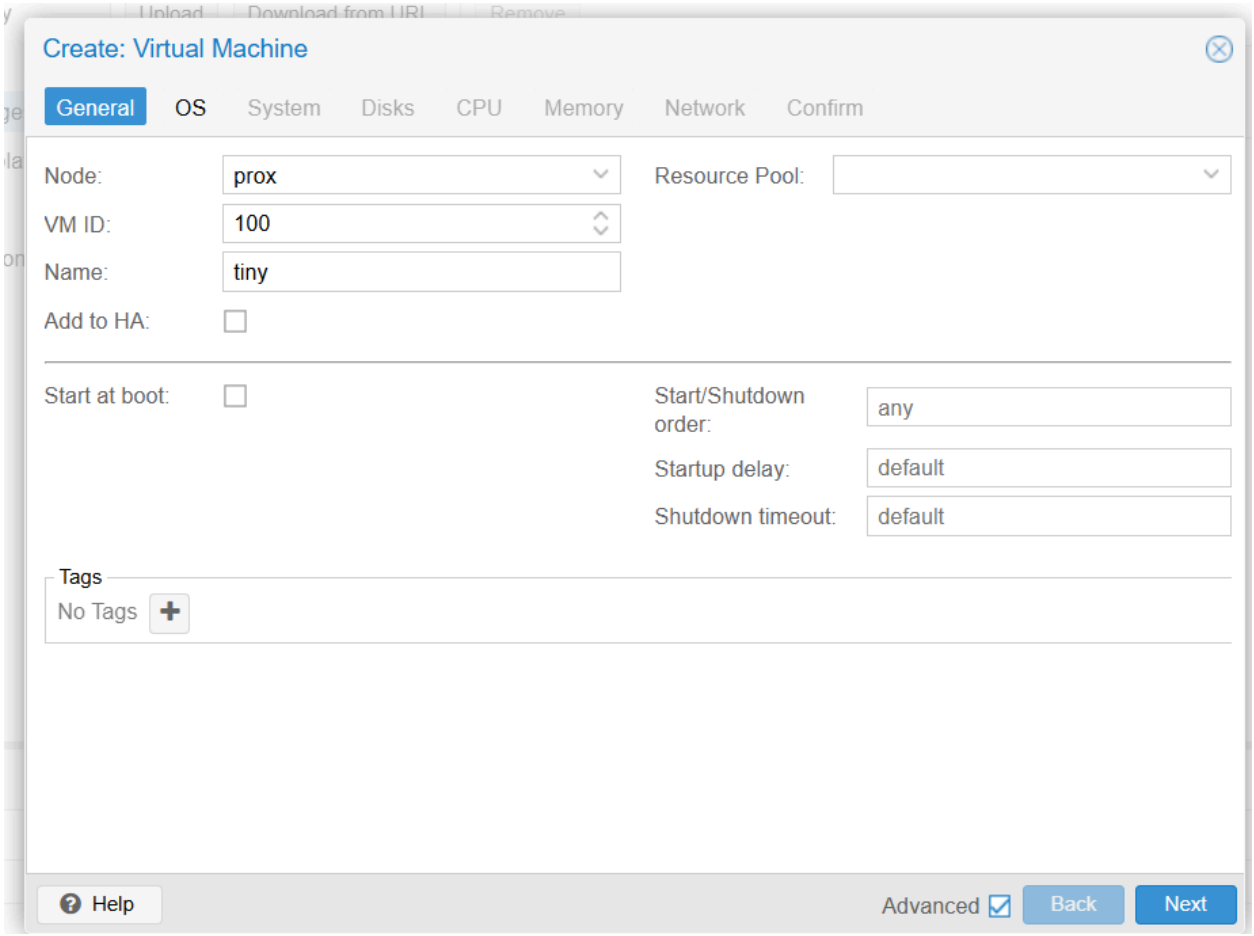
6. Select your **TinyCore ISO** file and click **Upload**.



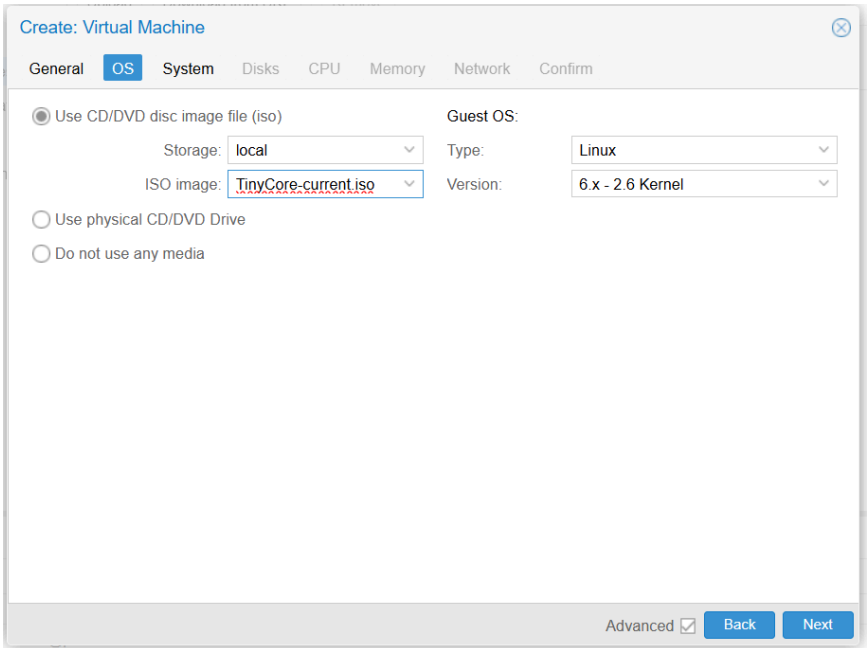
7. A dialog box will appear indicating "Task OK". Close this dialog once the upload is complete.

Phase 4: Provisioning a Virtual Machine (IaaS)

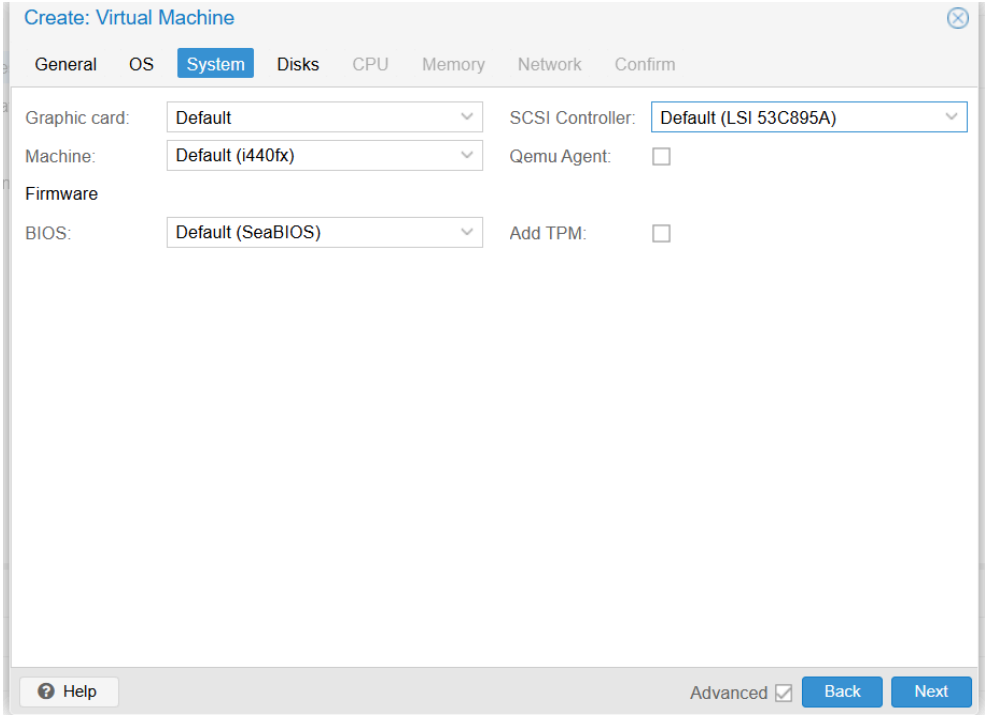
1. In the top right corner of the Proxmox dashboard, click the **Create VM** button.
2. **General Tab:** Enter a name for the VM (e.g., "tiny") and click **Next**.



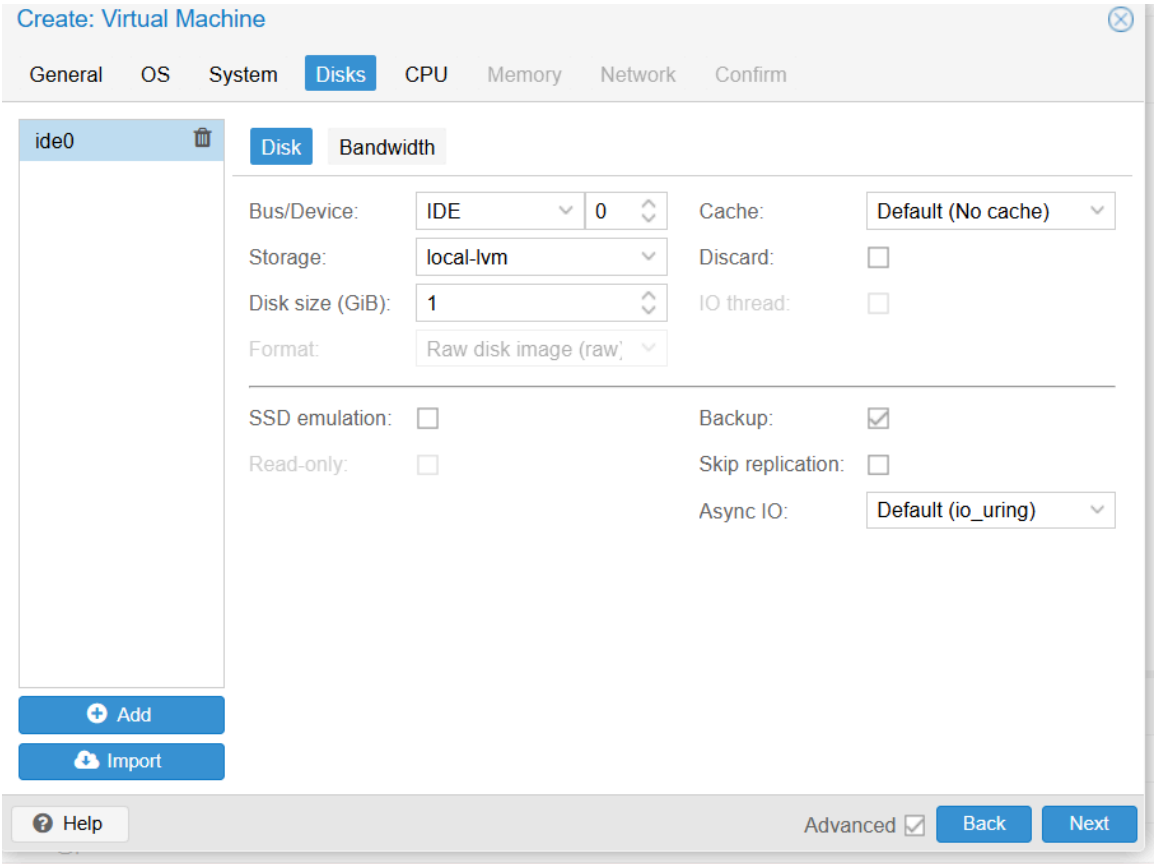
3. **OS Tab:** Select the uploaded **TinyCore ISO** file.



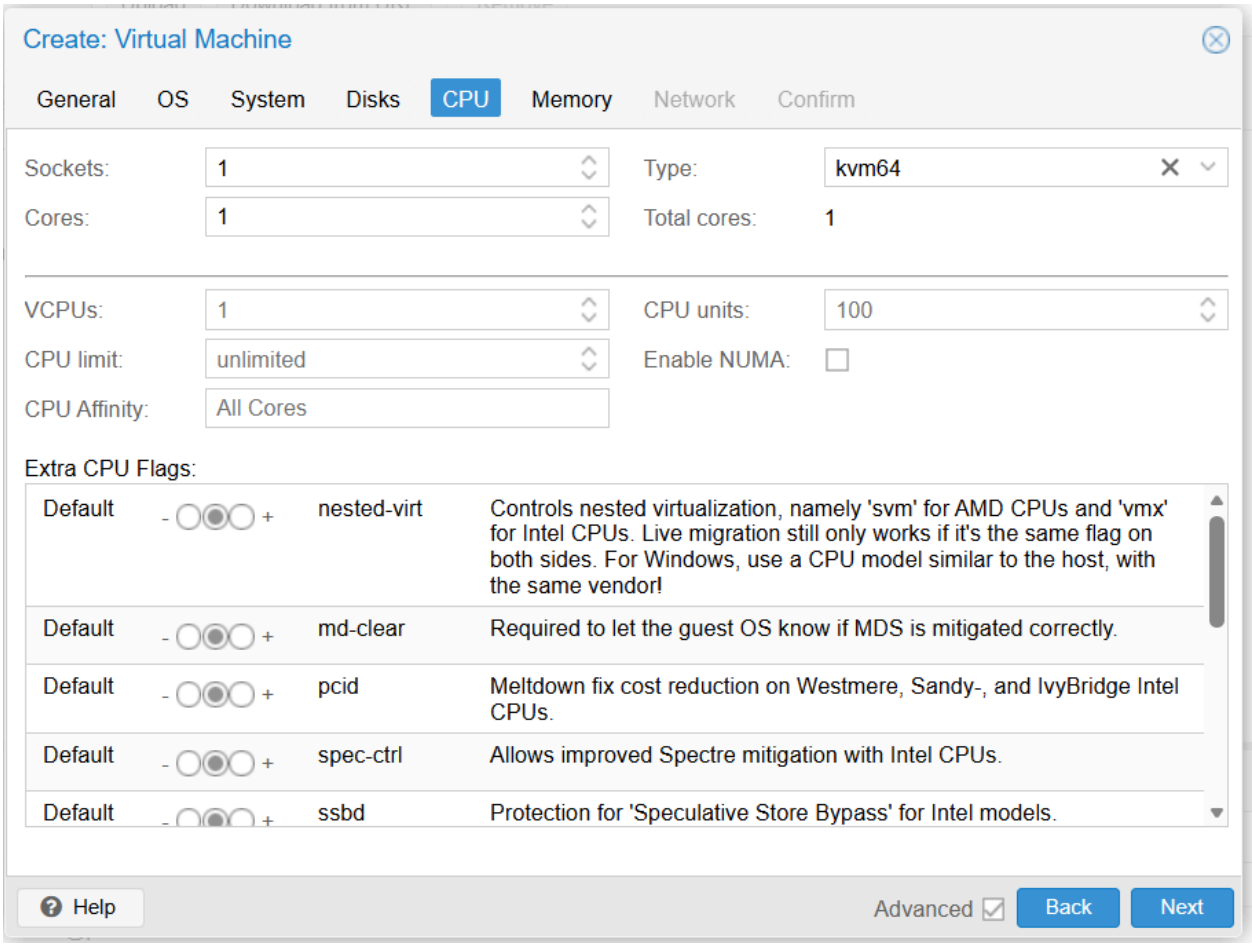
4. **System Tab:** Ensure the SCSI Controller is set to the default (**LSI 53C895A**).



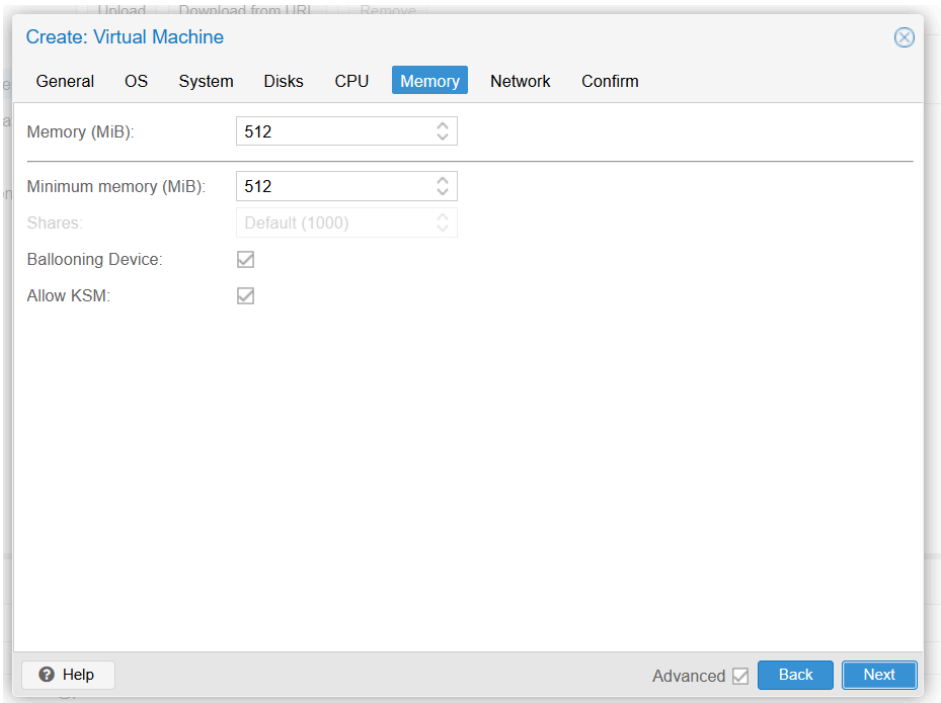
5. **Disks Tab:**
   - Change the Bus/Device to **IDE**.
   - Set the Disk size to **1 GB** (or 512 MB).



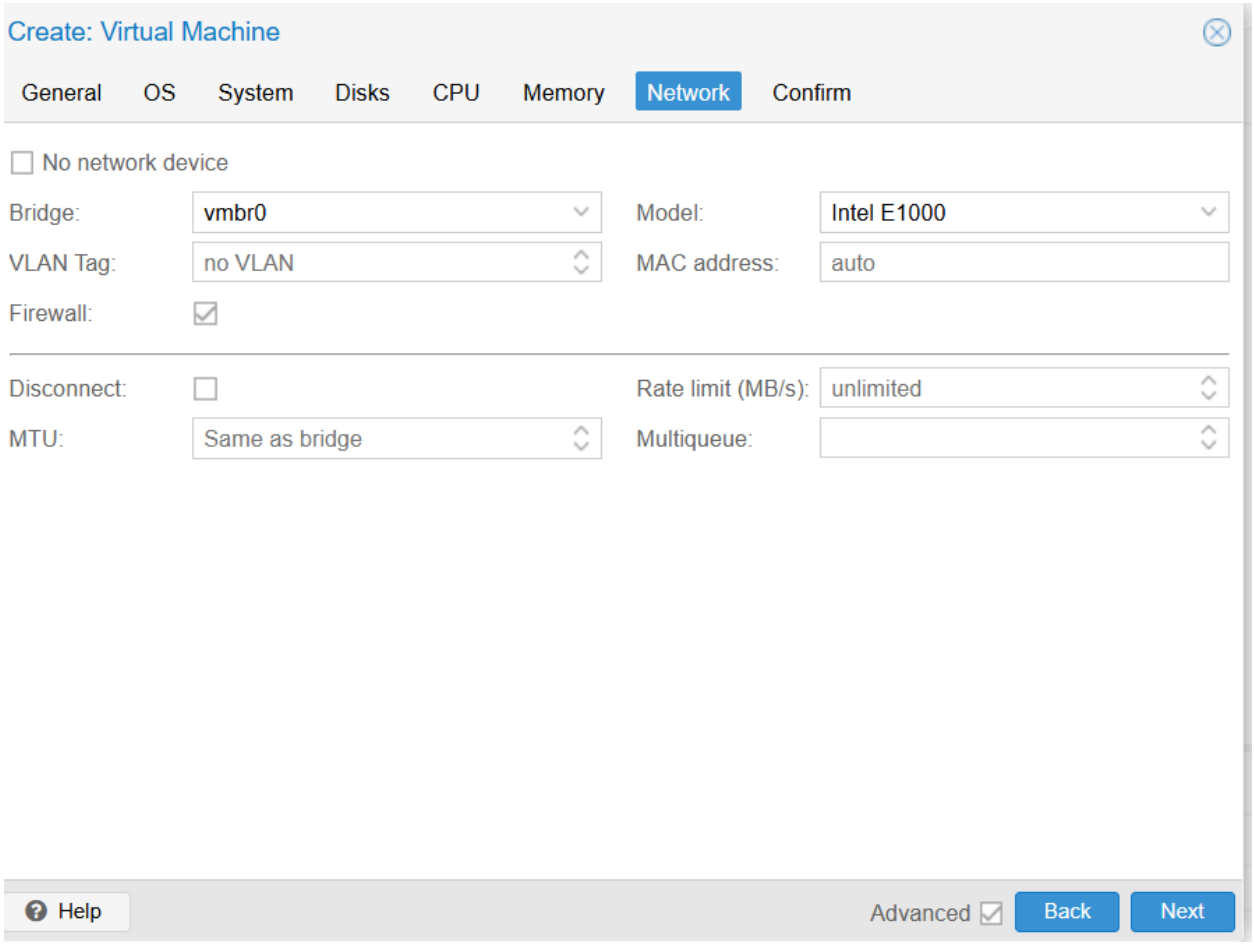
6. **CPU Tab:** Select the CPU type as **"kvm64"**.



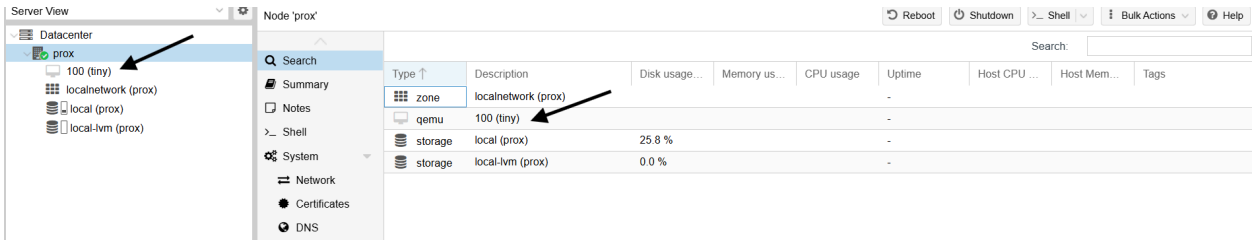
7. **Memory Tab:** Set the memory allocation to **512 MB**.



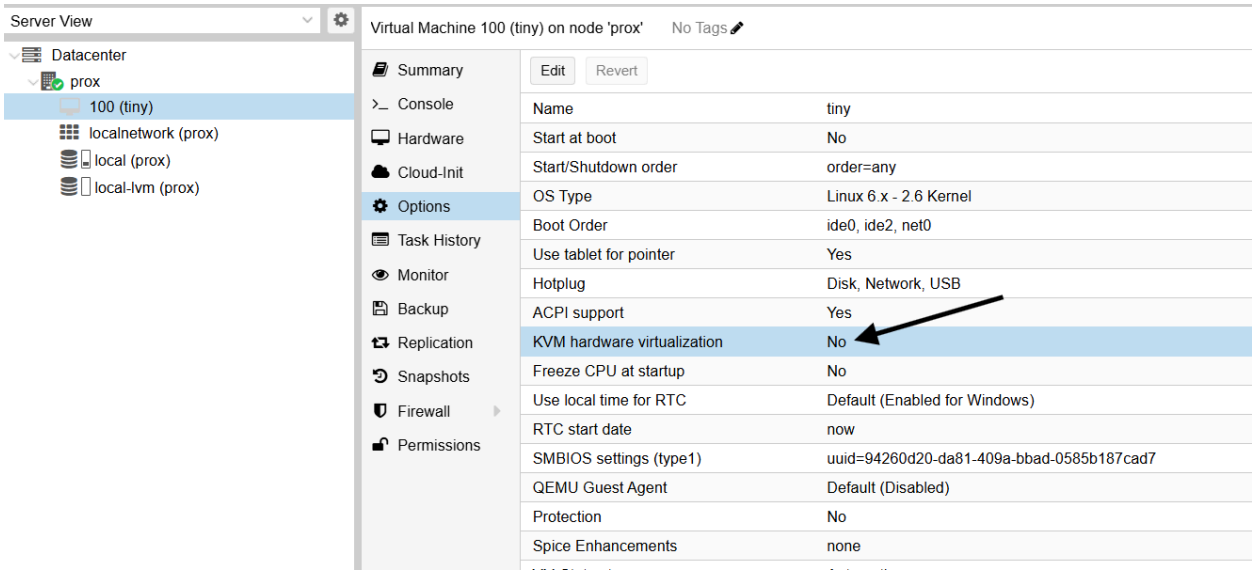
8. **Network Tab:** Select the model as **Intel E1000**, then click **Next** and **Finish** to create the VM.



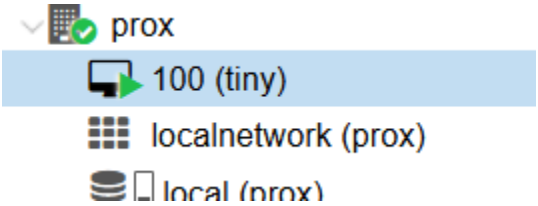
9. Locate your newly created VM (e.g., "tiny") in the left-hand navigation pane and select it.



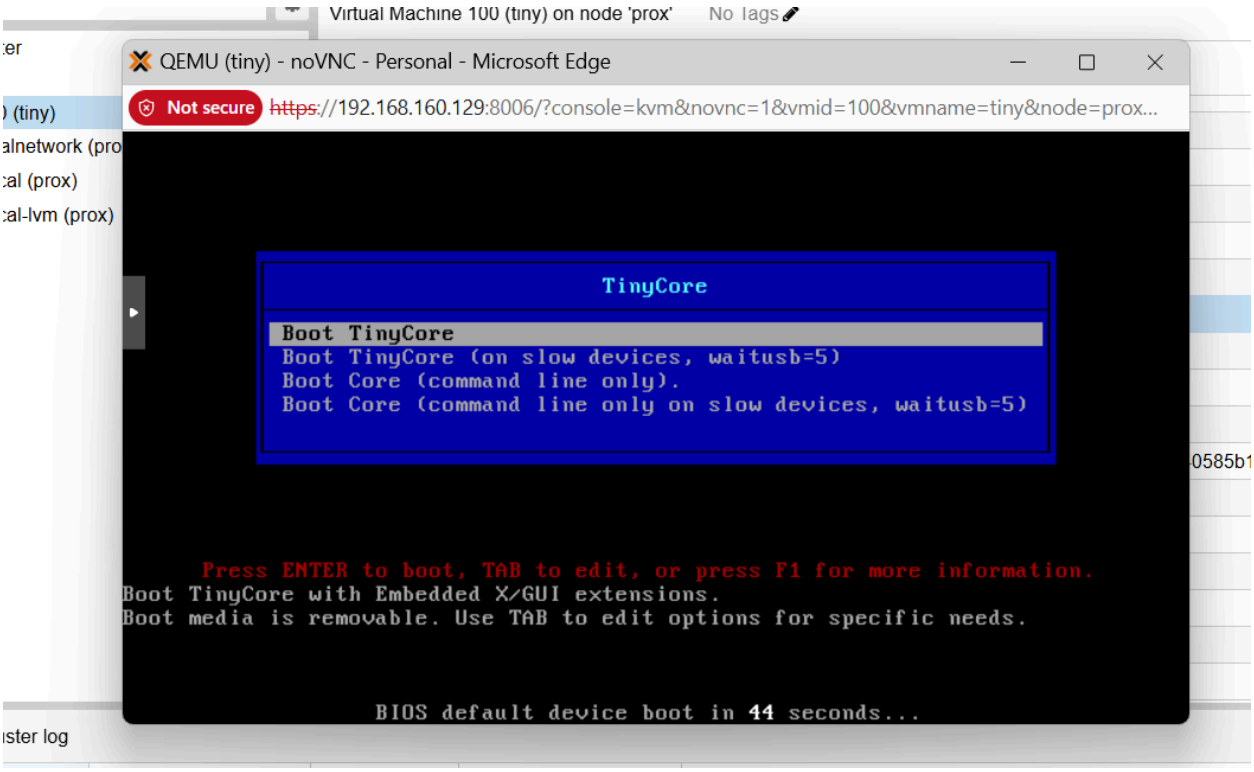
10. Go to the **Options** menu for that VM.
11. Locate the **KVM hardware virtualization** option and set it to **No**.



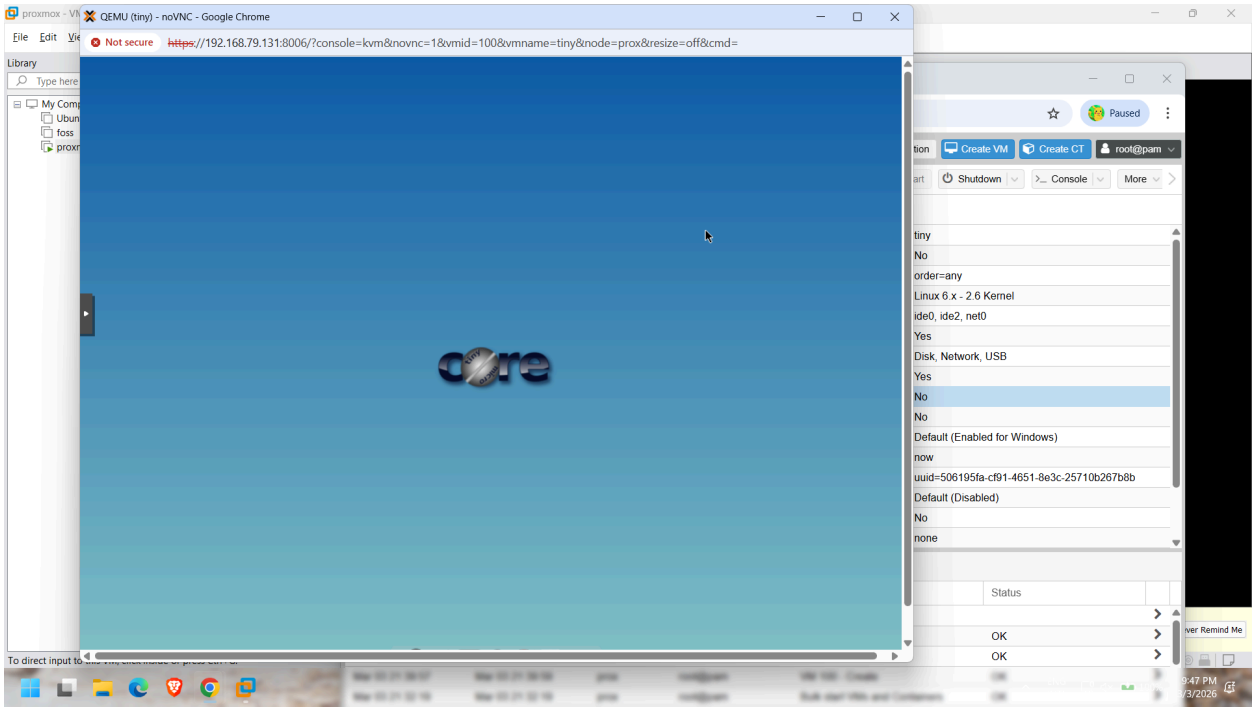
12. Click **Start VM** (located in the top right corner area).
13. If successful, a small green play button will appear next to the VM icon.
14. Double-click the VM icon to open the console view.



15. Once the VM display opens, you will see a "Boot TinyCore" prompt. Press **Enter** and wait for the OS to load



16. You've successfully created a cloud VM



Practical No. 9: Simple Workflow Application using AWS Flow Framework

1. Aim

To develop an application using the AWS Flow Framework that includes a simple workflow. The workflow calls an activity to print "Hello, World!" to the console. This demonstrates the basic usage of the AWS Flow Framework, including defining contracts, implementation of activities, workflow coordination logic, and worker programs to host them.

2. Prerequisites & Requirements

- **Development Environment:** Java Development Kit (JDK) and a build tool like Maven or Gradle.
- **Libraries:** AWS SDK for Java and AWS Flow Framework.
- **Cloud Access:** AWS credentials configured for programmatic access.

3. Procedure

Step 1: Set Up AWS Flow Framework dependencies Include the necessary dependencies in your project using Maven. Add the following to your `pom.xml`:

```XML
<dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>aws-sdk-java</artifactId>
</dependency>
<dependency>
    <groupId>software.amazon.aws</groupId>
    <artifactId>aws-flow-sdk</artifactId>
    <version>2.x.x</version>
</dependency>
```

Step 2: Define Contracts Define the contracts for your workflow and activity interfaces.

Java

```
public interface HelloWorldActivities {

    void printHelloWorld();

}



public interface HelloWorldWorkflow {

    void executeWorkflow();

}
```

Step 3: Implement Activities Implement the activities defined in the `HelloWorldActivities` interface, which represent the actual work that needs to be done.

Java

```
public class HelloWorldActivitiesImpl implements HelloWorldActivities {

    @Override

    public void printHelloWorld() {

        System.out.println("Hello, World!");

    }

}
```

Step 4: Implement Workflow Implement the workflow logic defined in the `HelloWorldWorkflow` interface to orchestrate the activities.

Java

```
import com.amazonaws.services.simpleworkflow.flow.annotations.Asynchronous;

import com.amazonaws.services.simpleworkflow.flow.annotations.Execute;

public class HelloWorldWorkflowImpl implements HelloWorldWorkflow {

   private final HelloWorldActivities activities = new HelloWorldActivitiesImpl();
```

```
    @Override

    @Execute(name = "HelloWorldWorkflow.executeWorkflow")

    public void executeWorkflow() {

        activities.printHelloWorld();

    }

}
```

Step 5: Implement Worker Programs Create worker programs to host the workflow and activity implementations.

```
import com.amazonaws.services.simpleworkflow.flow.ActivityWorker;

import com.amazonaws.services.simpleworkflow.flow.WorkflowWorker;

public class WorkerStarter {

    public static void main(String[] args) {

        String domain = "your-domain-name";

        String taskList = "your-task-list";

        // Create activity worker

        ActivityWorker activityWorker = new ActivityWorker(new
AmazonSimpleWorkflowClient(), domain, taskList);

        activityWorker.addActivitiesImplementation(new
HelloWorldActivitiesImpl());

        activityWorker.start();

        // Create workflow worker

        WorkflowWorker workflowWorker = new WorkflowWorker(new
AmazonSimpleWorkflowClient(), domain, taskList);


workflowWorker.addWorkflowImplementationType(HelloWorldWorkflowImpl.clas
s);

        workflowWorker.start();

    }

}
```

4. Output / Observation

- The worker programs successfully authenticated with Amazon Web Services.
- The workflow successfully registered and orchestrated the activity, resulting in "Hello, World!" being printed to the console execution logs.

5. Conclusion

We successfully implemented a cloud-based workflow using the AWS Flow Framework. By defining strict contracts and separating the activity implementation from the workflow orchestration logic, we demonstrated how distributed tasks can be managed and executed programmatically via AWS.

Practical No. 8

1. Aim

To implement FOSS-Cloud Platform as a Service (PaaS) functionality by deploying applications with integrated runtime environments and database services using a Java-based Platform Console.

2. Prerequisites & Requirements

- **Host Environment:** A system with Java Development Kit (JDK) installed.
- **Software:** Java Compiler (`javac`) and Java Runtime Environment (`java`).
- **Source Code:** `PAASConsole.java` and `FOSSPlatform.class`.
- **Hardware:** Sufficient resources to simulate a multi-instance application deployment.

3. Procedure

Step 1: Preparing the PaaS Environment

1. Ensure the `PAASConsole.java` file is present in your working directory.
2. Open a terminal or command prompt and navigate to the directory containing the file.
3. Compile the Java source code:

```
import java.util.Scanner;
class FOSSPlatform {
    String connection="Default-PAAS", correctUser="admin",
correctPass="cloud123", appName, runtime, database;
    int instances, storage;
    Scanner sc = new Scanner(System.in);
    void openPlatform() { System.out.println("\nConnecting to FOSS
Platform...\nPAAS Environment Loaded"); }
    void changeConnection() { System.out.print("\nEnter Platform
Connection Address: "); connection = sc.next();
System.out.println("Connected to: " + connection); }
    boolean authentication() {
        System.out.println("\n=== PLATFORM AUTHENTICATION ===");
        System.out.print("Username: "); String u = sc.next();
        System.out.print("Password: "); String p = sc.next();
        if(u.equals(correctUser) && p.equals(correctPass)) {
System.out.println("Login Successful"); return true; }
        else { System.out.println("Authentication Failed"); return false;
}
    }
    void deployApplication() {
        System.out.println("\n=== DEPLOY APPLICATION ===");
        System.out.print("Enter Application Name: "); appName =
sc.next();
```

```
        System.out.println("Select Runtime Environment:\n1. Java
(Tomcat)\n2. Node.js\n3. Python (Django)\n4. PHP");
        System.out.print("Choose (1-4): "); int choice = sc.nextInt();
        switch(choice) {
            case 1: runtime = "Java (Tomcat)"; break;
            case 2: runtime = "Node.js"; break;
            case 3: runtime = "Python (Django)"; break;
            case 4: runtime = "PHP"; break;
            default: runtime = "Java (Tomcat)"; break;
        }
        System.out.print("Number of Instances: "); instances =
sc.nextInt();
        System.out.println("Application Deployed Successfully");
    }
    void addDatabase() {
        System.out.println("\n=== ADD DATABASE SERVICE ===\nAvailable
Databases:\n1. MySQL\n2. PostgreSQL\n3. MongoDB\n4. Redis");
        System.out.print("Choose (1-4): "); int choice = sc.nextInt();
        switch(choice) {
            case 1: database = "MySQL"; break;
            case 2: database = "PostgreSQL"; break;
            case 3: database = "MongoDB"; break;
            case 4: database = "Redis"; break;
            default: database = "MySQL"; break;
        }
        storage = 5; System.out.println(database + " Database Added
Successfully");
    }
    void allocateStorage() { System.out.print("\nAdd Storage (GB): ");
storage += sc.nextInt(); System.out.println("Storage Allocated
Successfully"); }
    void showResult() {
        System.out.println("\n===== FINAL PLATFORM STATUS =====");
        System.out.println("Connection: "+connection+"\nApp Name:
"+appName+"\nRuntime: "+runtime+"\nDatabase: "+database+"\nInstances:
"+instances+"\nStorage: "+storage+" GB\nPlatform Deployment Completed");
    }
}
public class PAASConsole {
    public static void main(String[] args) {
        FOSSPlatform p = new FOSSPlatform(); p.openPlatform();
p.changeConnection();
        if(p.authentication()) { p.deployApplication(); p.addDatabase();
p.allocateStorage(); p.showResult(); }
        else { System.out.println("Access Denied"); }
    }
}
```

Step 2: Accessing the FOSS-Cloud Platform Console

1. Execute the compiled class to launch the FOSS-Cloud PaaS management simulator:

   java PAASConsole

2. The console will initialize the connection and load the platform environment.

Step 3: Platform Connection and Authentication

1. When prompted, enter the **Platform Connection Address** (e.g., `paas.cloud.local`).
2. Provide the administrator credentials for the platform portal:
   - **Username:** `admin`
   - **Password:** `cloud123`
3. Upon successful authentication, the system grants access to application deployment services.

Step 4: Application Deployment and Runtime Configuration

1. Enter the **Application Name** (e.g., `CloudStoreApp`).
2. Select the required **Runtime Environment** from the provided options:
   - (e.g., `Java (Tomcat)`, `Node.js`, `Python (Django)`, or `PHP`)
3. Specify the number of **Application Instances** to deploy for horizontal scaling (e.g., `3`).

Step 5: Database and Storage Service Configuration

1. Select an available **Database Service** to integrate with the application (e.g., `MySQL` or `PostgreSQL`).
2. Specify the amount of additional **Storage (GB)** to allocate for the application's persistent data (e.g., `10`).
3. The platform will finalize the configuration and display the complete deployment status.

---

4. Output / Observation

- The `PAASConsole` successfully managed the lifecycle of a cloud-native application deployment.
- The platform automatically configured the selected runtime and database service.
- A final status report was generated, summarizing the App Name, Runtime, Database, Instances, and total Storage allocated.

```
PS C:\Users\tembu\OneDrive\Desktop\cc practical\Practical_08> java .\PAASConsole.java

Connecting to FOSS Platform...
PAAS Environment Loaded

Enter Platform Connection Address: 192.168.0.0
Connected to: 192.168.0.0

=== PLATFORM AUTHENTICATION ===
Username: admin
Password: cloud123
Login Successful

=== DEPLOY APPLICATION ===
Enter Application Name: new
Select Runtime Environment:
1. Java (Tomcat)
2. Node.js
3. Python (Django)
4. PHP
Choose (1-4): 1
Number of Instances: 1
Application Deployed Successfully

=== ADD DATABASE SERVICE ===
Available Databases:
1. MySQL
2. PostgreSQL
3. MongoDB
4. Redis
Choose (1-4): 1
MySQL Database Added Successfully

Add Storage (GB): 1
Storage Allocated Successfully

===== FINAL PLATFORM STATUS =====
Connection: 192.168.0.0
App Name: new
Runtime: Java (Tomcat)
Database: MySQL
Instances: 1
Storage: 6 GB
Platform Deployment Completed
PS C:\Users\tembu\OneDrive\Desktop\cc practical\Practical_08>
```

5. Conclusion

We successfully implemented the Platform as a Service (PaaS) functionality within the FOSS-Cloud environment. We demonstrated the ability to provision isolated application runtimes, scale instances, and integrate database services through a centralized management interface.

Practical No. 10: Implementation of OpenStack with User and Private Network Creation

1. Aim

To implement OpenStack with user and private network creation.

2. Prerequisites & Requirements

- **Operating System:** A Linux Virtual Machine running Ubuntu 18.04.
- **Hardware:** The virtual machine must have at least 4GB of memory allocated.
- **Network:** A stable and proper internet connection is essential for downloading the repository packages.

3. Procedure

Step 1: Update the Ubuntu System Open the terminal and ensure the system repositories and installed packages are completely up to date.

```
sudo apt update

sudo apt -y upgrade

sudo apt -y dist-upgrade
```

After the update completes, reboot the system to apply the changes.

```
sudo reboot
```

Step 2: Create a Dedicated "Stack" User DevStack must be run as a regular, non-root user that has sudo privileges enabled. Create a new user specifically named "stack":

```
sudo useradd -s /bin/bash -d /opt/stack -m stack
```

Assign password-less sudo privileges to this newly created user:

```
echo "stack ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/stack
```

Switch over to the stack user account to continue the installation:

```
sudo su stack
```

Step 3: Install Git Ensure Git is installed on the system to allow downloading of the DevStack repository.

```
sudo apt install git -y
```

Step 4: Download OpenStack (DevStack) Clone the DevStack installation files directly from the official GitHub repository.

```
Sudo git clone https://git.openstack.org/openstack-dev/devstack
```

Step 5: Create the Configuration File Navigate into the newly downloaded DevStack directory:

```
cd devstack
```

Create a `local.conf` file to store the administrative passwords and define the host system's IP address. You can find your system's IP by running the `ifconfig` or `ip addr` command.

```
[[local|localrc]]

# Password for KeyStone, Database, RabbitMQ and Service

ADMIN_PASSWORD=StrongAdminSecret

DATABASE_PASSWORD=$ADMIN_PASSWORD

RABBIT_PASSWORD=$ADMIN_PASSWORD

SERVICE_PASSWORD=$ADMIN_PASSWORD



# Host IP

HOST_IP=192.168.56.103
```

Step 6: Execute the Installation Start the automated DevStack installation script.

```
./stack.sh
```

This script will install various OpenStack components, including Compute (Nova), Image Service (Glance), Identity (Keystone), Block Storage (Cinder), and the Horizon Dashboard. The installation process generally takes between 10 to 20 minutes depending on the internet speed.

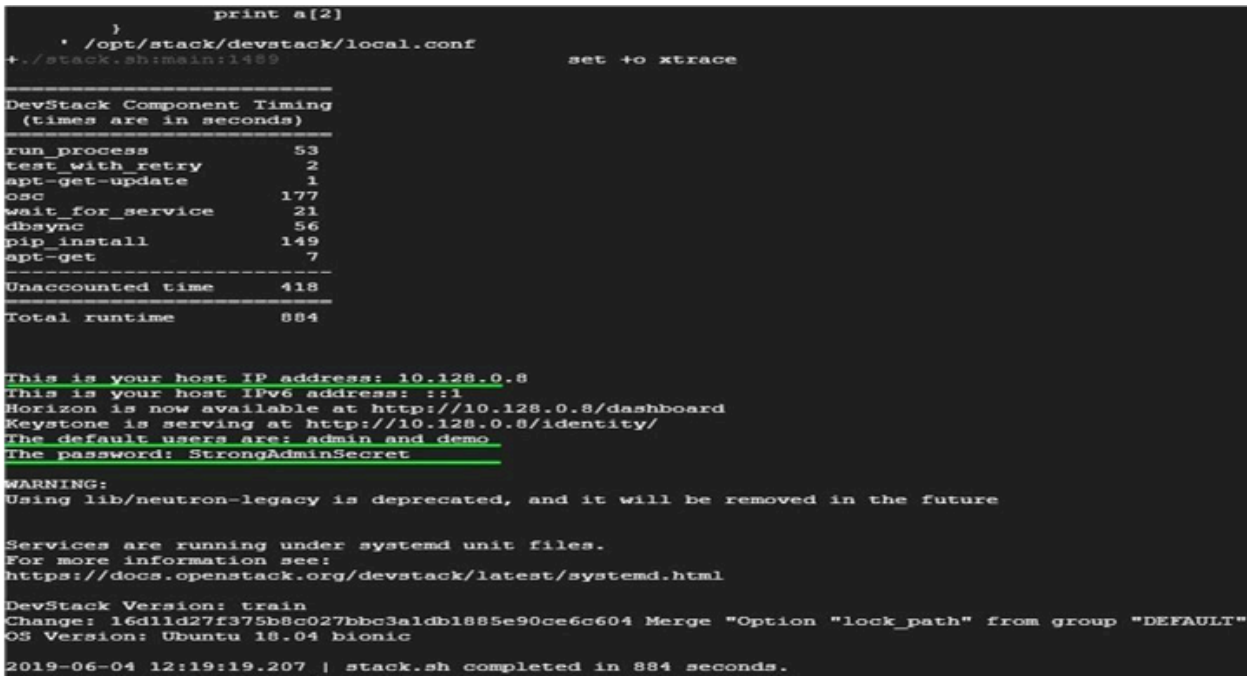
Step 7: Access the OpenStack Dashboard Once the installation finishes, the terminal will output an access link. Open a web browser and navigate to the Horizon URL: `http://<IP`

Address>/dashboard. Log in using the default username (admin or demo) and the password you configured in Step 5 (StrongAdminSecret).

Step 8: Provision an Instance From the OpenStack management web console dashboard, navigate to the **Instances** menu on the left side of the page. Click on **Launch Instances**, fill in the required configuration fields, and deploy the new virtual machine instance.

---

4. Output / Observation



```
                print a[2]
        }
    ' /opt/stack/devstack/local.conf
+./stack.sh:main:1489                          set +o xtrace

=========================
DevStack Component Timing
 (times are in seconds)
=========================
run_process             53
test_with_retry          2
apt-get-update           1
osc                    177
wait_for_service        21
dbsync                  56
pip_install            149
apt-get                  7
-------------------------
Unaccounted time       418
=========================
Total runtime          884


This is your host IP address: 10.128.0.8
This is your host IPv6 address: ::1
Horizon is now available at http://10.128.0.8/dashboard
Keystone is serving at http://10.128.0.8/identity/
The default users are: admin and demo
The password: StrongAdminSecret

WARNING:
Using lib/neutron-legacy is deprecated, and it will be removed in the future


Services are running under systemd unit files.
For more information see:
https://docs.openstack.org/devstack/latest/systemd.html

DevStack Version: train
Change: 16d11d27f375b8c027bbc3a1db1885e90ce6c604 Merge "Option "lock_path" from group "DEFAULT"
OS Version: Ubuntu 18.04 bionic

2019-06-04 12:19:19.207 | stack.sh completed in 884 seconds.
```

openstack.
admin
Project
API Access
Compute
Overview
Instances
Images
Key Pairs
Server Groups
Volumes
Network
Admin
Identity
Project / Compute / Overview
Overview
Limit Summary
Compute
Instances
Used 0 of 10
VCPUs
Used 0 of 20
RAM
Used 0Bytes of 50GB
Volume
Volumes
Used 0 of 10
Volume Snapshots
Used 0 of 10
Volume Storage
Used 0Bytes of 1000GB
Network